

Deploying Bizlance Automation — A Guide for Total Beginners

If you’ve never used GitHub, a terminal, or an API key before, this is written for you. Every step explains WHAT you’re doing and WHY, not just the click-path.

Part 0 — What you’re actually building, in plain English

You have a folder of Python files (little programs). Each one does one job: “write a blog post,” “post to Instagram,” “send an email.” Right now they just sit on your computer, doing nothing — they need somewhere to *live* that runs them automatically, every day, forever, without your laptop needing to be on.

That “somewhere” is **GitHub Actions** — a free feature of GitHub (a website for storing code) that can run your Python files on a schedule, in the cloud, automatically.

So the whole deployment is really just 3 kinds of steps, repeated: 1. Put the code on GitHub (once) 2. Get “keys” (passwords, basically) from each service you’re using (Google, Brevo, OpenAI, etc.) and give them to GitHub so your scripts can use them 3. Tell GitHub to actually run things on schedule (flip one switch)

That’s it. Nothing here requires you to know how to code.

Part 1 — Install what you need

1. A GitHub account Go to github.com → Sign up → free account, just needs an email.

2. Windows/Mac: GitHub Desktop (skip this if you’re on Android) Go to desktop.github.com → download → install → sign in with the GitHub account you just made. This avoids the terminal entirely.

Android: nothing to install yet — Part 2B below covers the Android-specific tool (Termux) as part of that walkthrough.

That’s genuinely all you need. No coding knowledge required for either path.

Part 2 — Get the code onto GitHub

Which path applies to you: if you’re on Windows or Mac, use Part 2A below (GitHub Desktop, no terminal). If you’re on Android with no computer, skip to Part 2B — GitHub Desktop doesn’t

exist for Android, so that path uses a terminal app instead (it's genuinely not hard, just different).

Part 2A — Windows/Mac, using GitHub Desktop

1. Unzip `bizlance-automation.zip` on your computer — you should see a folder called `bizlance-automation` with files like `README.md`, a `scripts` folder, etc.
2. Open **GitHub Desktop**.
3. Click **File** → **Add Local Repository**.
4. Browse to and select the `bizlance-automation` folder you unzipped.
5. It will say “This directory does not appear to be a Git repository” — click “**create a repository**” right there in that message.
6. Click **Publish repository** (top of the window). Uncheck “Keep this code private” only if you’re fine with it being public — private is safer, keep it checked.
7. Click **Publish**.

Done — your code is now on GitHub. You’ll see it at `github.com/YOUR-USERNAME/bizlance-automation`.

Whenever a file changes later: GitHub Desktop will show the changed files on the left, you type a one-line summary at the bottom, click **Commit**, then click **Push origin** at the top. That’s the entire “save my changes to the cloud” workflow, forever.

Part 2B — Android only, using Termux (a terminal app)

GitHub Desktop only exists for Windows/Mac. On Android, the equivalent tool is a terminal app called **Termux** — it sounds more technical than it is; you’re just typing short commands one line at a time.

1. **Install Termux** from F-Droid.org (search “Termux,” install the app) — not the Play Store version, which is outdated and often broken. Also install **ZArchiver** (free, Play Store) if your Files app can’t unzip on its own.
2. Unzip `bizlance-automation.zip` using your Files app or ZArchiver — it’ll land in your Downloads folder.
3. Open Termux and type each of these, pressing Enter after each one:

```
pkg update
pkg install git unzip -y
termux-setup-storage
```

The last command pops up an Android permission request — tap **Allow**.

4. Move into the unzipped folder:

```
cd ~/storage/downloads/bizlance-automation
```
5. **Create the empty repo on GitHub first**, from your phone’s browser (easier than Termux for this step): go to `github.com` → tap + → **New repository** → name it `bizlance-automation` → keep it **Private** → do NOT check “Add a README” → **Create**

repository.

6. **Get a Personal Access Token** (this is what lets Termux “log in” to push code — GitHub no longer accepts your actual password for this): github.com → tap your profile picture → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token** → check the **repo** checkbox → **Generate token** → **copy it immediately** — you cannot view it again after leaving the page.

7. Back in Termux, push the code:

```
git init
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/bizlance-automation.git
git add .
git commit -m "Initial commit"
git push -u origin main
```

Replace YOUR-USERNAME with your actual GitHub username in that remote add line. When it asks for a username, type your GitHub username. When it asks for a password, **paste the token from step 6** instead.

Done — same result as Part 2A, just via command line instead of clicking buttons.

Whenever a file changes later: reopen Termux, cd back into the folder (step 4), then run:

```
git add .
git commit -m "update"
git push
```

That’s the entire “save my changes to the cloud” workflow on Android, forever.

Part 3 — Understanding “secrets” (this is the only slightly fiddly part)

Your scripts need passwords to work — a password for Google, a password for the email service, etc. You never put passwords directly in your code (anyone could see them). Instead, GitHub has a vault called **Secrets** where you paste each password once, and your code says “go get the password named X from the vault” instead of containing it directly.

Where the vault is: on your repo’s GitHub page → **Settings** tab → left sidebar **Secrets and variables** → **Actions** → green button **New repository secret**.

You’ll repeat this “paste a name, paste a value, save” action once for every item below. Let’s go get each password.

Part 4 — Getting each key, one at a time

Do these in order. Skip any section for a feature you’re not using yet (e.g. skip Meta/social until

you’ve done App Review) — the system is built so missing pieces just quietly skip that one feature instead of breaking everything else.

4a. Blog posting (GitHub Pages) — no external account needed

Your blog lives right in this repo. One-time toggle, no signup: 1. Go to your repo on github.com → **Settings** tab → **Pages** (left sidebar). 2. Under “Build and deployment,” Source: **Deploy from a branch**. 3. Branch: **main**, folder: **/docs** → **Save**.

That’s it — no secret to add. Once the daily blog job runs for the first time, your blog will be live at <https://YOUR-USERNAME.github.io/bizlance-automation/>.

4b. Cold email sending (Gmail — your own account, no new signup)

1. Go to myaccount.google.com/security (sign in with the Gmail you’ll send from).
2. Find **2-Step Verification** — turn it on if it isn’t already (needed before Google will let you create an App Password).
3. Go to myaccount.google.com/apppasswords.
4. Under “App name,” type “Bizlance” → **Create**.
5. Google shows you a 16-character password (looks like abcd efgh ijkl mnop) — copy it. This is NOT your normal Gmail password — it’s a special one just for this automation.

Add these secrets: - GMAIL_ADDRESS = your Gmail address - GMAIL_APP_PASSWORD = the 16-character password from step 5 (spaces don’t matter)

4c. Lead list (a plain file in your repo — no Google Sheets needed)

1. On your repo’s GitHub page, click **leads.csv** → click the pencil (✎) icon to edit.
2. Add one row per lead, following the existing header row exactly:
company_name, contact_name, email, industry, website, segment, status, last_sent_date
segment must be exactly provider or buyer. Leave status and last_sent_date blank — the script fills those in automatically.
3. Scroll down, click **Commit changes**.

That’s the entire setup — no account, no API key, no sharing step. Same idea for `partnerships.csv` if you want weekly partnership outreach running (columns: org_name, contact_name, email, org_type, website, status, last_sent_date).

4d. Social images (OpenAI + imgbb)

1. Go to platform.openai.com → sign up/sign in → **Settings** → **Billing** → add a payment method (this one costs money per image, unlike everything else so far).
2. **API keys** (left menu) → **Create new secret key** → copy it immediately (you can’t view it again later).
3. Go to api.imgbb.com → sign up free → your API key is shown on that page → copy it.

Add these secrets: - OPENAI_API_KEY = the key from step 2 - IMGBB_API_KEY = the key from

step 3

4e. Social posting itself (Meta: Instagram, Facebook, Threads) — do this LAST, it's the slow one

This needs Meta's App Review approval first (1-3 weeks), covered in the separate `META_APP_REVIEW.md` file. **Threads needs its own separate approval** even though it's the same Meta family — different permissions, same submission round if you request both together (see Section 7 of that file). Come back to this section once approved. When it is, you'll get tokens from Meta's dashboard — add them as: - `META_PAGE_ACCESS_TOKEN`, `META_IG_BUSINESS_ID`, `META_PAGE_ID` (Instagram + Facebook) - `THREADS_ACCESS_TOKEN`, `THREADS_USER_ID` (Threads)

Until then, just don't add these — those jobs will fail harmlessly and everything else keeps running.

4f. X (Twitter) — skipped on purpose

X's API has no free posting tier (pay-per-use since Feb 2026, ~\$0.015/post — cheap but not free). This system stays draft-only for X: it writes post text to a file for you to copy-paste manually, no key needed, no cost. Nothing to set up here unless you later decide to pay for real auto-posting.

Part 5 — Test everything before trusting the schedule

1. On your repo's GitHub page, click the **Actions** tab (top).
2. You'll see a list of workflows on the left: "Daily Blog Post," "Daily Business Outreach," etc.
3. Click one → click the **Run workflow** button (top right of that list) → **Run workflow** (confirm).
4. Wait ~30-60 seconds, refresh, click into the run that appears → click the job name → you'll see line-by-line logs, like a diary of exactly what the script did.
5. Green checkmark = it worked. Red X = something failed — click into the failed step, read the error message (it usually tells you plainly what's wrong, e.g. "Missing required environment variable: `BREVO_API_KEY`" means you typo'd a secret name).

Do this for each workflow once. Only trust the automatic daily schedule once each one has gone green at least once manually.

Part 6 — What happens now, forever, without you touching anything

Once secrets are in and you've done Part 5: - Every day, the blog posts itself. - Twice a day, Instagram/Facebook post + story themselves (once Meta approves). - Every weekday, up to

30+30 cold emails send themselves. - Every Monday, newsletter/carousel/partnership drafts get made.

You do NOT need to open your laptop, run any command, or click anything for this to keep happening. GitHub's servers run it, on schedule, in the cloud.

Part 7 — The only things that still need YOU, occasionally

- **Refill the Leads sheet** every couple weeks (new rows, using free tools like Hunter.io — details in `OUTREACH_SETUP.md`).
 - **Check** `dm_drafts.jsonl` in your repo occasionally and manually send any reply you like on Instagram — this stays human-controlled on purpose.
 - **Meta App Review** — a one-time approval, not repeating.
 - **Glance at the Actions tab** every so often to make sure things are still green, not red.
-

Troubleshooting cheat sheet

Error mentions...

“Missing required environment variable: X”

401 / “Unauthorized”

“leads.csv not found”

Blog job runs green but the site 404s

Social job fails, everything else fine

Usually means...

You forgot to add secret X, or typed its name wrong (secret names must match EXACTLY, including capital letters)

A key/token is wrong, expired, or copy-pasted with an extra space

You haven't added the file yet, or it's not in the repo root — check step 4c

GitHub Pages takes 1-2 minutes to go live after first enabling it, and after the very first post — give it a minute and refresh

Expected until Meta App Review is approved — not a bug

If you get stuck on any single step, tell me exactly which step number and paste the error text — that's all I need to help you past it.